

Examen Final — Parte 1

Práctica CI/CD

Informe de Reflexión

Estudiante: nnyez
Materia: Sistemas Operativos
Fecha: 29 de julio de 2026

Índice

1. Parte I — Construcción y Despliegue

1.1. Pipeline Base (Docker + CI/CD + Kubernetes)

1.1.1. Dockerfile Multi-Stage

El `Dockerfile` consta de dos etapas:

builder: Imagen `node:18-alpine3.21`. Instala dependencias con `npm ci` y ejecuta `npm test`. Si los tests fallan, el build falla.

runtime: Imagen `node:18-alpine3.21` mínima. Ejecuta `apk --no-cache upgrade`, elimina `npm` (no necesario en runtime), y copia solo `node_modules/`, `public/`, `db.js`, `server.js`, `data/`.

Para construir y probar localmente:

```
1 docker build -t inventario-app ./inventario-app
2 docker run -d -p 3000:3000 --name inventario inventario-app
3 curl localhost:3000/health      # {"status":"ok"}
4 curl localhost:3000/version     # {"version":"v1","color":"blue"}
5 curl localhost:3000/api/products # Lista de 3 productos seed
6 docker stop inventario && docker rm inventario
```

1.1.2. Pipeline CI/CD (GitHub Actions)

El archivo `.github/workflows/ci-cd.yml` define dos jobs encadenados:

- **build-test:** `npm ci + npm test` en Node.js 18.
- **build-push (needs: build-test):** Construye y publica la imagen en `ghcr.io` con tags `sha-xxxxxx` y `latest`.

Incluye además escaneo de seguridad con Trivy (`aquasecurity/trivy-action@v0.36.0`) que falla el pipeline si encuentra vulnerabilidades **CRITICAL**.

El pipeline corre exitosamente y publica la imagen en `ghcr.io/nnyez/inventario-app`.

1.1.3. Despliegue en Minikube (RollingUpdate)

Se creó el namespace `inventario-app` y se aplicaron los manifiestos:

```
1 kubectl create namespace inventario-app
2 kubectl apply -f k8s/deployment-config.yaml -n inventario-app
3 kubectl apply -f k8s/service.yaml -n inventario-app
4 kubectl apply -f k8s/deployment.yaml -n inventario-app
```

El `deployment.yaml` configura:

- 2 réplicas.
- `RollingUpdate` con `maxUnavailable: 1` y `maxSurge: 1`.
- `readinessProbe: /health, initialDelaySeconds: 5, periodSeconds: 10, failureThreshold: 6`.

- `livenessProbe: /health, initialDelaySeconds: 15, periodSeconds: 20.`
- `secretKeyRef` para cargar `API_KEY` desde el `Secret`.

Verificación exitosa:

```
1 kubectl rollout status deployment/inventario-app -n inventario-app
2 kubectl port-forward -n inventario-app service/inventario-app 8080:80
3 curl localhost:8080/health      # {"status":"ok"}
4 curl localhost:8080/version     # {"version":"v1","color":"blue"}
5 curl localhost:8080/api/products # 3 productos seed
```

1.2. Segunda Estrategia de Despliegue: Canary

1.2.1. Justificación de la elección

Se eligió **Canary** sobre Blue-Green por las siguientes razones:

- **Proporcionalidad nativa de Services:** Kubernetes distribuye el tráfico proporcional a la cantidad de pods que coinciden con el selector. Con 4 réplicas estables y 1 canary, el $\sim 80\%$ del tráfico va a la versión estable y $\sim 20\%$ a la nueva.
- **Riesgo mínimo:** Un error en la nueva versión afecta solo al $\sim 20\%$ de los usuarios.
- **Eficiencia:** Canary requiere solo 1 réplica extra (5 pods) frente a 8 pods de Blue-Green.
- **Rollback inmediato:** Basta con escalar el canary a 0 réplicas.

1.2.2. Implementación

Manifiestos en `k8s/canary/`:

- `stable.yaml`: 4 réplicas, `APP_VERSION=v1`, `APP_COLOR=blue`.
- `new.yaml`: 1 réplica, `APP_VERSION=v2`, `APP_COLOR=green`.
- `service.yaml`: Selector `app: inventario-app` (balancea entre ambos).

1.2.3. Demostración de reparto de tráfico

Se ejecutaron 30 peticiones desde dentro del clúster:

```
1 kubectl run test --image=alpine/curl:latest --rm -it --restart=Never --
  /bin/sh -c '
2   for i in $(seq 1 30); do wget -qO- http://inventario-app/version; echo
    ""; done
3 '
```

Resultado: 29 respuestas **blue** (**v1**) y 1 respuesta **green** (**v2**), confirmando la distribución proporcional.

1.3. Componentes Adicionales de Buenas Prácticas

Se implementaron los tres componentes (2 obligatorios + 1 extra):

1.3.1. Componente 1: Manejo de Secretos

Se creó un Secret de Kubernetes con una API_KEY ficticia:

```
1 kubectl create secret generic inventario-app-secrets -n inventario-app \
2 --from-literal=API_KEY="sk-test-1234567890"
```

El deployment.yaml lo consume vía secretKeyRef:

```
1 env:
2 - name: API_KEY
3   valueFrom:
4     secretKeyRef:
5       name: inventario-app-secrets
6       key: API_KEY
```

La credencial **no está versionada en Git** (el .gitignore excluye datos sensibles y los comandos de creación están documentados con un placeholder).

1.3.2. Componente 2: Escaneo de Seguridad con Trivy

El pipeline incluye Trivy que escanea la imagen después de publicarla y falla si encuentra vulnerabilidades CRITICAL:

```
1 - name: Security scan with Trivy
2   uses: aquasecurity/trivy-action@v0.36.0
3   with:
4     image-ref: ghcr.io/nnyez/inventario-app:latest
5     exit-code: '1'
6     severity: CRITICAL
```

Durante el desarrollo, Trivy detectó 2 CRITICALs (CVE-2026-31789 en OpenSSL y CVE-2026-59873 en tar), que fueron corregidas actualizando la imagen base a node:18-alpine3.21 y eliminando npm del runtime.

1.3.3. Componente 3: Readiness con Arranque Lento

La aplicación soporta la variable STARTUP_DELAY_SECONDS. Cuando está activa, /health responde 503 durante los primeros N segundos:

```
1 // server.js
2 const STARTUP_DELAY_SECONDS = parseInt(process.env.STARTUP_DELAY_SECONDS
3   || '0', 10);
4 if (STARTUP_DELAY_SECONDS > 0) {
5   const elapsed = (Date.now() - serverStartTime) / 1000;
6   if (elapsed < STARTUP_DELAY_SECONDS) {
7     return res.status(503).json({ status: 'starting', message: '
8     Arrancando...' });
9   }
10 }
```

Se activó en el ConfigMap (STARTUP_DELAY_SECONDS: "15") y se verificó que el readinessProbe tolera el retraso (initialDelaySeconds: 5, periodSeconds: 10, failureThreshold: 6 ⇒ hasta 60s de tolerancia). Los eventos de Kubernetes confirmaron:

```
1 30s Warning Unhealthy Readiness probe failed: HTTP probe failed with
   statuscode: 503
```

Tras 15 segundos, el pod se marcó como Ready.

¿Qué pasaría si en vez de ajustar el probe se aumenta el número de réplicas?

Aumentar réplicas no resuelve el problema: cada pod individualmente tarda en arrancar, y sin un `readinessProbe` tolerante, Kubernetes mataría y recrearía el pod en un ciclo infinito (`CrashLoopBackOff`) porque nunca se marca como “Ready”.

1.4. Observación sobre Persistencia de Datos

Se creó un producto vía API:

```
1 curl -X POST localhost:8080/api/products \
2   -H "Content-Type: application/json" \
3   -d '{"name":"Producto nuevo","sku":"NEW-001","stock":5,"price":99.99}'
```

Luego se forzó la eliminación de un pod:

```
1 kubectl delete pod -n inventario-app -l app=inventario-app --limit=1
2 kubectl rollout status deployment/inventario-app -n inventario-app
```

Tras la recreación, el catálogo contenía solo 3 productos (los seed originales). El producto nuevo desapareció.

Causa: La base de datos es un archivo JSON local (`data/products.json`) en el filesystem efímero del pod. Al recrearse el pod, se usa la imagen original que contiene solo los 3 productos seed. No hay volumen persistente (PVC) ni base de datos externa.

Conclusión: Es el comportamiento esperado. Para entornos productivos se necesitaría un PVC o una base de datos externa.

2. Parte II — Métricas DORA

2.1. Lead Time for Changes

Tiempo entre el commit y su despliegue en el clúster:

Cambio	Commit	Despliegue	Lead Time
Pipeline base	2026-07-29 04:26	2026-07-29 04:35	~9 min
Canary + Secret	2026-07-29 05:06	2026-07-29 05:07	~1 min
STARTUP_DELAY	2026-07-29 05:07	2026-07-29 05:08	~1 min

2.2. Deployment Frequency

Métrica	Valor
Total pipelines ejecutados	10
Pipelines exitosos	5
Días de trabajo	1 (2026-07-29)
Frecuencia	10 deploys/día

2.3. Change Failure Rate (CFR)

Métrica	Valor
Total despliegues	10
Fallos que requirieron corrección	5
Tasa de fallo	50 %

2.4. Reflexión sobre las Métricas

La alta tasa de fallo (50 %) refleja la etapa inicial de configuración del pipeline, donde se corrigieron problemas de autenticación, versión de acciones, caché de Docker y vulnerabilidades de seguridad.

Según la tabla de niveles DORA:

- **CFR 50 %** → Nivel **Bajo (Low)**.
- **Frecuencia 10 deploys/día** → Nivel **Alto (High)**.
- **Lead Time ~1-9 min** → Nivel **Élite (Elite)**.

Con un pipeline maduro y estable, se espera reducir el CFR por debajo del 15 % (nivel Alto).

3. Problemas Reales Encontrados

Problema	Causa	Solución
Pipeline fallaba en build-push	<code>trivy-action@0.28.0</code> no existe	Cambiar a <code>@v0.36.0</code>
Build fallaba por caché	<code>cache-to=type=gha</code> incompatible con driver docker	Eliminar caché
Error “denied: installation not allowed”	<code>GITHUB_TOKEN</code> sin <code>packages:write</code>	Agregar <code>permissions</code> al workflow
Trivy encontraba CRITICALs	<code>node:18-alpine</code> con OpenSSL y tar obsoletos	Actualizar a <code>18-alpine3.21</code> , <code>apk upgrade</code> , eliminar npm
Pods no arrancaban	Secret <code>inventario-app-secrets</code> no existía	<code>kubectl create secret</code>
Service no balanceaba con port-forward	<code>port-forward</code> no distribuye tráfico entre pods	Probar desde dentro del clúster

4. Conclusiones

1. Se construyó exitosamente un pipeline de CI/CD completo con Docker multi-stage, GitHub Actions, y despliegue Kubernetes en Minikube.
2. Se implementó la estrategia Canary para despliegues graduales, aprovechando la proporcionalidad nativa de los Services de Kubernetes.
3. Se incorporaron tres componentes de buenas prácticas: manejo de secretos, escaneo de seguridad con Trivy, y readiness con arranque lento.
4. Se verificó que la base de datos JSON local no persiste entre recreaciones de pod, comportamiento esperado sin PVC.
5. Se calcularon métricas DORA propias: lead time ~1-9 min, frecuencia 10 deploys/día, CFR 50 %.
6. Se documentaron 6 problemas reales encontrados y sus soluciones, demostrando el proceso de aprendizaje práctico.